



Advanced TypeScript Usage

Intersection Types

Type Guards

Discriminated Unions

Function Overloads

Constant Types with "as const"

Sometimes you might want to force TypeScript

to infer a bit of a more narrow type

and you can do so by adding as const here

```
8  
9   store.id = 5;  
10  store.isOpen = false;  
11  // store.name = 'Max';  
12  
13  let roles = ['admin', 'guest', 'editor'] as const;
```

And this simply tells TypeScript hey,

```
let roles = ['admin', 'guest', 'editor'] as const;
```

when inferring the type for this variable,

be as narrow as possible or do it as narrowly as possible.

Don't infer a broad type like string array,

```
10 store.isOpen = false;
11 // s
12 let roles: readonly ["admin", "guest", "editor"];
13 let roles = ['admin', 'guest', 'editor'] as const;
```

```
2
3 let roles = ['admin', 'guest', 'editor'] as const;
4 roles.push('max');
```

can't push some other string into this array,

Revising the "Record" Type

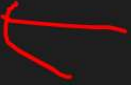
You might recall that "Record" type to tell typescript that you wanted to store some object in a given variable or constant or argument of a function. Record is so called generic type.

```
1 type DataStore = {  
2   [prop: string]: number | boolean;  
3 };  
4  
5 let someObj: Record<string, number | boolean>;  
6  
7 let store: DataStore = {};  
8  
9 // ...  
10  
11 store.id = 5;  
12 store.isOpen = false;  
13 // store.name = 'Max';  
14
```



The "satisfies" keyword

```
12  
13 let roles = ['admin', 'guest', 'editor'] as const;  
14 // roles.push('max');  
15 const firstRole = roles[0];  
16  
17 const dataEntries: Record<string, number> = {  
18   entry1: 0.51,  
19   entry2: -1.23  
20 };  
21  
22 // ...  
23  
24 dataEntries.entry3
```



We are not getting an error here even though, entry3 does not exist in 'dataEntries' object.

```
12
13 let roles = ['admin', 'guest', 'editor'] as const;
14 // roles.push('max');
15 const firstRole = roles[0];
16
17 const dataEntries = {
18   entry1: 0.51,
19   entry2: -1.23
20 };
21
22 // ...
23
24 dataEntries.entry3
```

error

If I remove this type assignment here and instead

```
let roles = ['admin', 'guest', 'editor'] as const;
// roles.push('max');
const firstRole = roles[0];

const dataEntries = {
  entry1: 0.51,
  entry2: -1.23
} satisfies Record<string, number>;

// ...

dataEntries.entry3
```

Satisfies also tells TypeScript

that data entries will hold a record type

with string properties and number of values.

But when using satisfies,

```

const dataEntries: = {
  entry1: 0.51,
  entry2: -1.23
} satisfies Record<string, number>;

// ...

dataEntries.entry3

```

instead of the colon notation here,

```

3 let roles = ['admin', 'guest', 'editor'] as const;
4 // roles.push('max');
5 const firstRole = roles[0];
6
7 const dataEntries = {
8   entry1: 0.51,
9   entry2: -1.23
10 } satisfies Record<string, number>;
11
12 // ...
13
14 dataEntries.entry3

```

TypeScript will actually take a look at the value you are

assigning to data entries.

```

let roles = ['admin', 'guest', 'editor'] as const;
// roles.push('max');
const firstRole = roles[0];
const dataEntries = {
  entry1: 0.51,
  entry2: -1.23
} satisfies Record<string, number>;

// ...

dataEntries.entry3

```

It will check whether that value fits this type here,

```

3 let roles = ['admin', 'guest', 'editor'] as const;
4 // roles.push('max');
5 const firstRole = roles[0];
6
7 const dataEntries = {
8   entry1: 0.51,
9   entry2: -1.23
0 } satisfies Record<string, number>;
1
2 // ...
3
4 dataEntries.entry3

```

if it's a valid type of value given the type

it will actually use the more specific type it can

infer from that value.

So put in other words, TypeScript will use the type you set

after the satisfies keyword

to check whether the actual value assigned

to data entries is of that type.

as a second step actually infer a more specific type from